CISS245: Advanced Programming
Quiz q6003

Name: eagraham1@cougars.ccis.edu Score:

Q1. A hunter lives in the Wumpus world, which is a 4-by-4 grid. He/she walks in the direction he/she is facing. Initially he/she is facing N (north). In each time step in Wumpus world, the hunter (1) randomly selects a direction, (2) randomly decides to walk one step (in his/her direction) or not, and (3) randomly decides to fire one arrow (in his/her direction) or not. Here's a test run:

```
+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
|H| | | |
+-+-+-+-+
Hunter direction: N

+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
| |H| | |
+-+-+-+-+
Hunter direction: E

+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
| |H| | |
+-+-+-+-+
Hunter direction: S

+-+-+-+-+
| | | | |
```

```
+-+-+-+-+
| |^| | |
+-+-+-+-+
| |H| | |
+-+-+-+-+
| | | | |
+-+-+-+-+
Hunter direction: N

+-+-+-+-+
| |^| | |
+-+-+-+-+
| | | | |
+-+-+-+-+
| | |H| |
+-+-+-+-+
| | | | |
+-+-+-+-+
Hunter direction: E

+-+-+-+-+
| | | | |
+-+-+-+-+
| | | | |
+-+-+-+-+
| | | |H|
+-+-+-+-+
| | | | |
+-+-+-+-+
Hunter direction: E
```

The ^ is the arrow flying in the N direction. You should draw the arrow using ^, v, >, < depending on the direction N,S,E,W (respectively) of flight.

Note that when the hunter fires his/her arrow, the arrow will fly in the direction of the hunter (at that point in time). The hunter can only fire one arrow (too bad). The hunter will also not walk into his arrow.

You are given this:

```
// file: WumpusWorld.h
#ifndef WUMPUSWORLD_H
#define WUMPUSWORLD_H

#include <iostream>

class WumpusWorld
{
public:
```

```
    void init();         // initialize so that hunter is at (3,0).
    void println();      // print according to the above format
    void move_hunter();  // 1. randomize a direction rand() % 4 where 0,1,2,3
                         //    are N,S,E,W for hunter.
                         // 2. randomize whether to walk or not
                         // 3. randomize whether to fire arrow or not
                         // Of course hunter must stay in the world.
                         // If hunter tries to move N but its row is 0,
                         // hunter stays put.
    void move_arrow();   // move arrow (if necessary)
private:
    char world_[4][4];
    int hunter_direction_;
    int arrow_state_;    // 0-N, 1-S, 2-E, 3-W, 4-not fired, 5-hit something
};
#endif
```

You want to make the hunter wonder around like this:

```
#include <iostream>
#include <ctime>
#include <cstdlib>
#include "WumpusWorld.h"

int main()
{
    srand((unsigned int) time(NULL));

    WumpusWorld ww;
    ww.init();
    ww.println();
    for (int i = 0; i < 50; ++i)
    {
        ww.move_hunter();
        ww.move_arrow();
        ww.println();
    }
    return 0;
}
```

Complete the following cpp file that contains the implementation of the methods declared in the WumpusWorld class.
Answer:

```
// file: WumpusWorld.cpp
int hunter_row, hunter_col;
int arrow_row, arrow_col;
int arrow_dir;
```

```
void WumpusWorld::init()
{
  for (int i = 0; i < 4; ++i)
  {
    for (int j = 0; j < 4; ++j)
    {
      world_[i][j] = ' ';
    }
  }

  hunter_row = 3;
  hunter_col = 0;
  hunter_direction_ = 0;

  world_[hunter_row][hunter_col] = 'H';

  arrow_state_ = 4;
}

void WumpusWorld::println()
{
  for (int i = 0; i < 4; ++i)
  {
    std::cout << "+-+-+-+-+" << std::endl;
    for (int j = 0; j < 4; ++j)
    {
      std::cout << "|" << world_[i][j];
    }
    std::cout << "|" << std::endl;
  }
  std::cout << "+-+-+-+-+" << std::endl;

  std::string dir;
  switch (hunter_direction_)
  {
    case 0: dir = "N"; break;
    case 1: dir = "S"; break;
    case 2: dir = "E"; break;
    case 3: dir = "W"; break;
  }
  std::cout << "Hunter direction: " << dir << std::endl;
}

void WumpusWorld::move_hunter()
{
  hunter_direction_ = rand() % 4;

  if (rand() % 2 == 1)
  {
    int new_row = hunter_row;
    int new_col = hunter_col;
```

```
    if (hunter_direction_ == 0 && hunter_row > 0) new_row--;
    else if (hunter_direction_ == 1 && hunter_row < 3) new_row++;
    else if (hunter_direction_ == 2 && hunter_col < 3) new_col++;
    else if (hunter_direction_ == 3 && hunter_col > 0) new_col--;

    if (world_[new_row][new_col] != '^' &&
    world_[new_row][new_col] != 'v' &&
    world_[new_row][new_col] != '<' &&
    world_[new_row][new_col] != '>')
    {
      world_[hunter_row][hunter_col] = ' ';
      hunter_row = new_row;
      hunter_col = new_col;
      world_[hunter_row][hunter_col] = 'H';
    }
  }

  if (arrow_state_ == 4 && rand() % 2 == 1)
  {
    arrow_state_ = 0;
    arrow_row = hunter_row;
    arrow_col = hunter_col;
    arrow_dir = hunter_direction_;

    switch (arrow_dir)
    {
      case 0: world_[arrow_row][arrow_col] = '^'; break;
      case 1: world_[arrow_row][arrow_col] = 'v'; break;
      case 2: world_[arrow_row][arrow_col] = '>'; break;
      case 3: world_[arrow_row][arrow_col] = '<'; break;
    }
  }
}

void WumpusWorld::move_arrow()
{
  if (arrow_state_ != 0) return;

  if (!(arrow_row == hunter_row && arrow_col == hunter_col))
  world_[arrow_row][arrow_col] = ' ';

  if (arrow_dir == 0) arrow_row--;
  else if (arrow_dir == 1) arrow_row++;
  else if (arrow_dir == 2) arrow_col++;
  else if (arrow_dir == 3) arrow_col--;

  if (arrow_row < 0 || arrow_row > 3 || arrow_col < 0 || arrow_col > 3)
  {
    arrow_state_ = 5;
    return;
```

```
  }

  switch (arrow_dir)
  {
    case 0: world_[arrow_row][arrow_col] = '^'; break;
    case 1: world_[arrow_row][arrow_col] = 'v'; break;
    case 2: world_[arrow_row][arrow_col] = '>'; break;
    case 3: world_[arrow_row][arrow_col] = '<'; break;
  }
}
```

Instructions

In the file `thispreamble.tex` look for

```
\renewcommand\AUTHOR{}
```

and enter your email address:

```
\renewcommand\AUTHOR{jdoe5@cougars.ccis.edu}
```

(This is not really necessary since alex will change that for you when you execute `make`.) In your bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

Enter your answers in `main.tex`. In the bash shell, execute "`make`" to recompile `main.pdf`. Execute "`make v`" to view `main.pdf`.

For each question, you'll see boxes for you to fill. For small boxes, if you see

```
1 + 1 = \answerbox{}.
```

you do this:

```
1 + 1 = \answerbox{2}.
```

`answerbox` will also appear in "true/false" and "multiple-choice" questions.

For longer answers that need typewriter font, if you see

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
\end{answercode}
```

you do this:

```
Write a C++ statement that declares an integer variable name x.
\begin{answercode}
int x;
\end{answercode}
```

`answercode` will appear in questions asking for code, algorithm, and program output. In this case, indentation and spacing is significant. For program output, I do look at spaces and newlines.

For long answers (not in typewriter font) if you see

```
What is the color of the sky?
\begin{answerlong}
\end{answerlong}
```

you can write

```
What is the color of the sky?
\begin{answerlong}
The color of the sky is blue.
\end{answerlong}
```

A question that begins with "T or F or M" requires you to identify whether it is true or false, or meaningless. "Meaningless" means something's wrong with the question and it is not well-defined. Something like "$1 + 2 = 4$" is either true or false (of course it's false). Something like "$1+_2 = 4$?" does not make sense.

When writing results of computations, make sure it's simplified. For instance write 2 instead of $1 + 1$.

Higher level classes.

For students beyond 245: You can put LaTeX commands in `answerlong`.

More examples of meaningless statements: Questions such as "Is $42 = 1+_2$ true or false?" or "Is $42 = \{2\}^{\{3\}}$ true or false?" does not make sense. "Is $P(42) = \{42\}$ true or false?" is meaningless because $P(X)$ is only defined if X is a set. For "Is $1 + 2 + 3$ true or false?", "$1 + 2 + 3$" is well–defined but as a "numerical expression", not as a "proposition", i.e., it cannot be true or false. Therefore "Is $1 + 2 + 3$ true or false?" is also not a well-defined question.

More examples of simplification: When you write down sets, if the answer is $\{1\}$, do not write $\{1, 1\}$. And when the values can be ordered, write the elements of the set in ascending order. When writing polynomials, begin with the highest degree term.

When writing a counterexample, always write the simplest.